File System

Dept. of Computer Science

Hanyang University

File Concept

- ‘Container’ for data and program
  - Contiguous logical address space that contains data or program

- Types:
  - Data
    - Numeric
    - Character
    - Binary
  - Program

File Structure

- None - sequence of words, bytes
- Simple record structure
  - Lines
  - Fixed length
  - Variable length
- Complex Structures
  - Formatted document
  - Relocatable load file
- Who decides:
  - Operating system
  - Program

File Attributes (File Metadata)

- **Name** – only information kept in human-readable form
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time**, **date**, **and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory (Folder) structure, which is maintained on the disk

File Operations

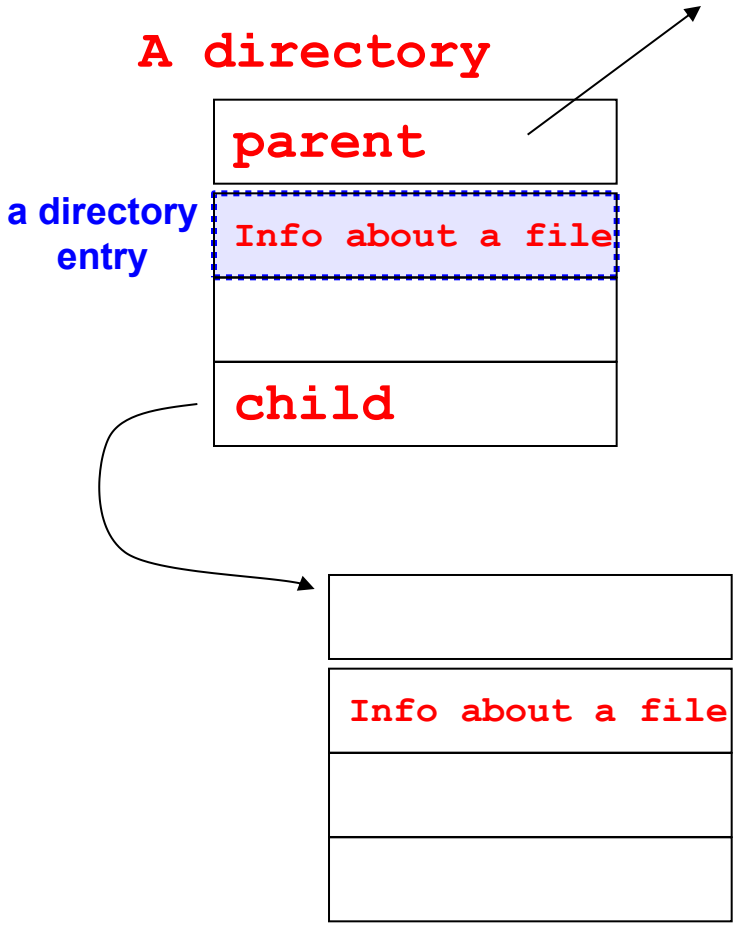
- create
- write
- read
- repositioning within file – file seek
- delete
- truncate
- open(F_i) – Copies file metadata from disk to memory
  (Search the directory structure to do this)
- close (F_i)

Directories

- Directories serve two purposes
  - For users, they provide a structured way to organize related files
  - For the file system, they provide a convenient naming interface which allows the implementation to hide details about where a file's particular data item is
- Most systems support multi-level directories, where a file name describes its path from a root through the directories to the file at the leaf
- Most systems have a *current* directory, from which names can be specified relatively, as opposed to absolutely from the root of the directory tree

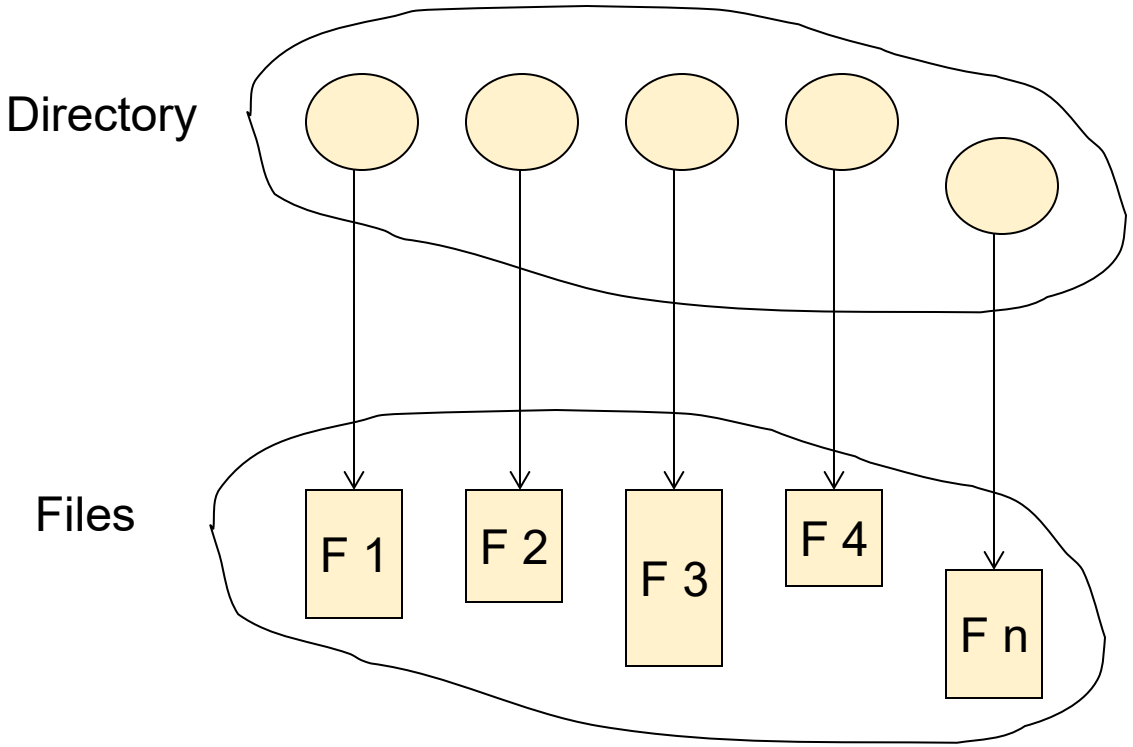
A Directory Entry

- A directory describes the logical information about a file
  - File name, size, type, location, protection, last access time
- This stuff is stored on disk in a data structure
  - That's why there may be limitations
- The operating system caches directory entries for recently accessed files in memory
  - Hopefully, the cache is kept consistent with the source data in disks
  - Otherwise, you can lose a file!



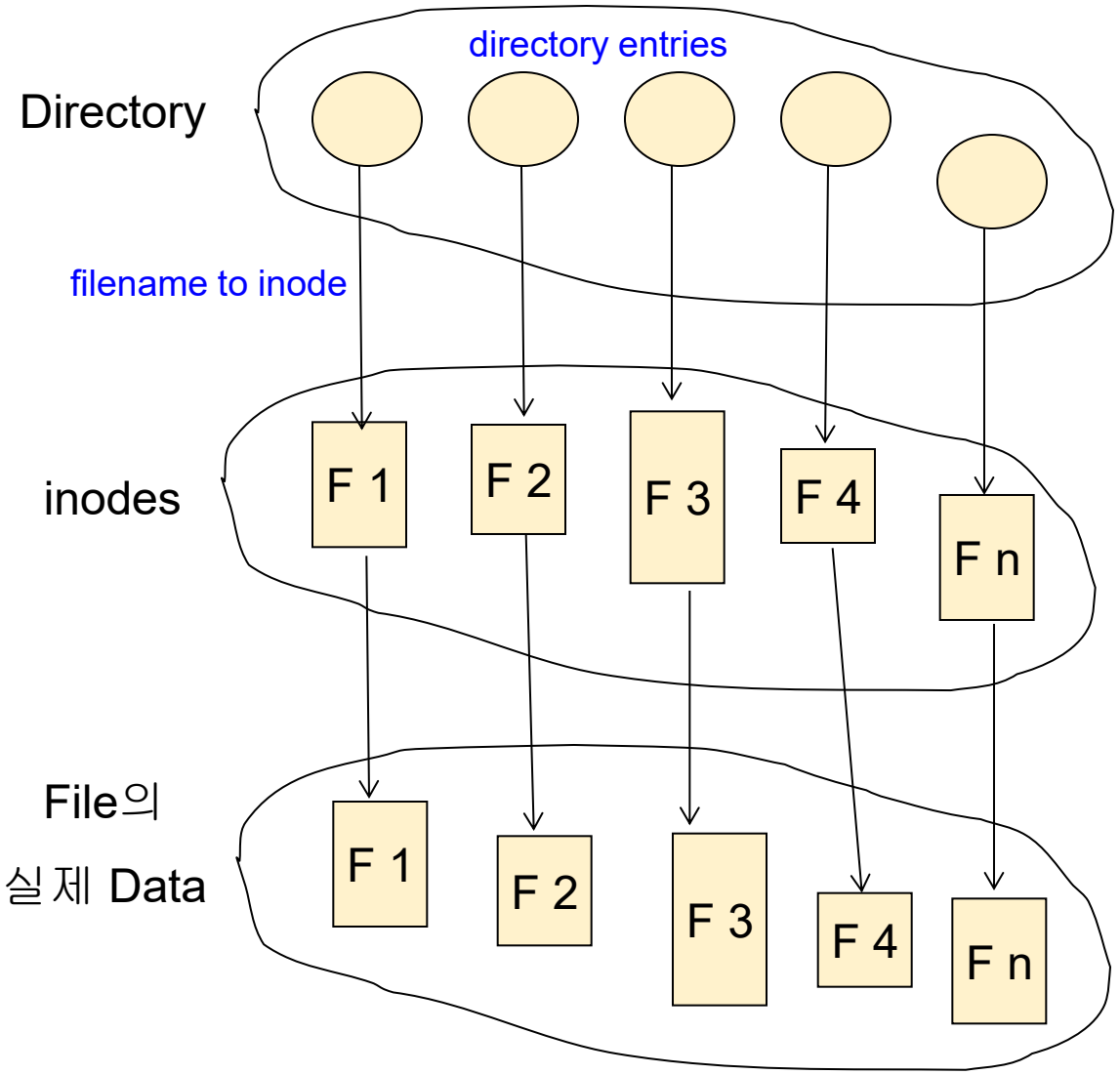
Directory Structure

- A collection of nodes containing information about files



Both the directory structure and the files reside on disk

Directory Structure (UNIX)



An entry contains a filename and an inode pointer for the file. (Note! There can be entries for subdirectories.)

An inode contains metadata for a file.

Metadata includes pointers to data blocks for the file

Directory Implementation

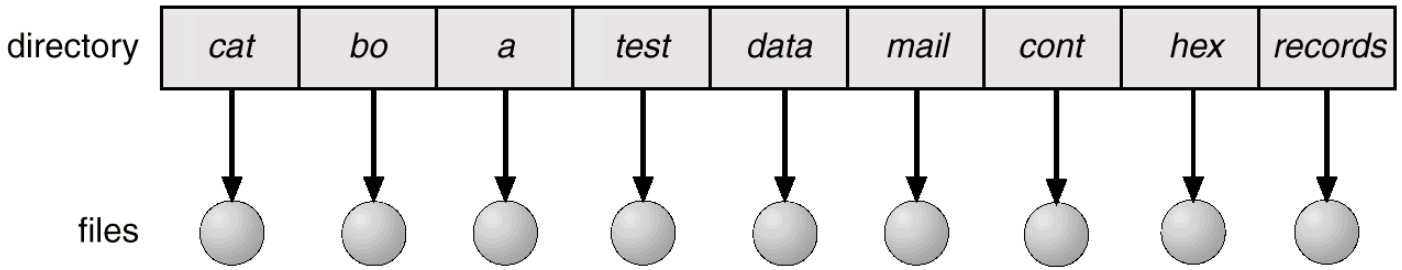
- Linear list of file names with metadata (or pointers to inodes in UNIX)
  - Simple to program
  - Linear search to find a particular entry, which is time-consuming
- Hash Table
  - Linear list (but has extra hash table)
  - Hash table converts (file name) to pointer to this file
  - Decreases directory search time
  - *collisions* – situations where two file names hash to the same location

Operations Performed on Directory

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

Single-Level Directory

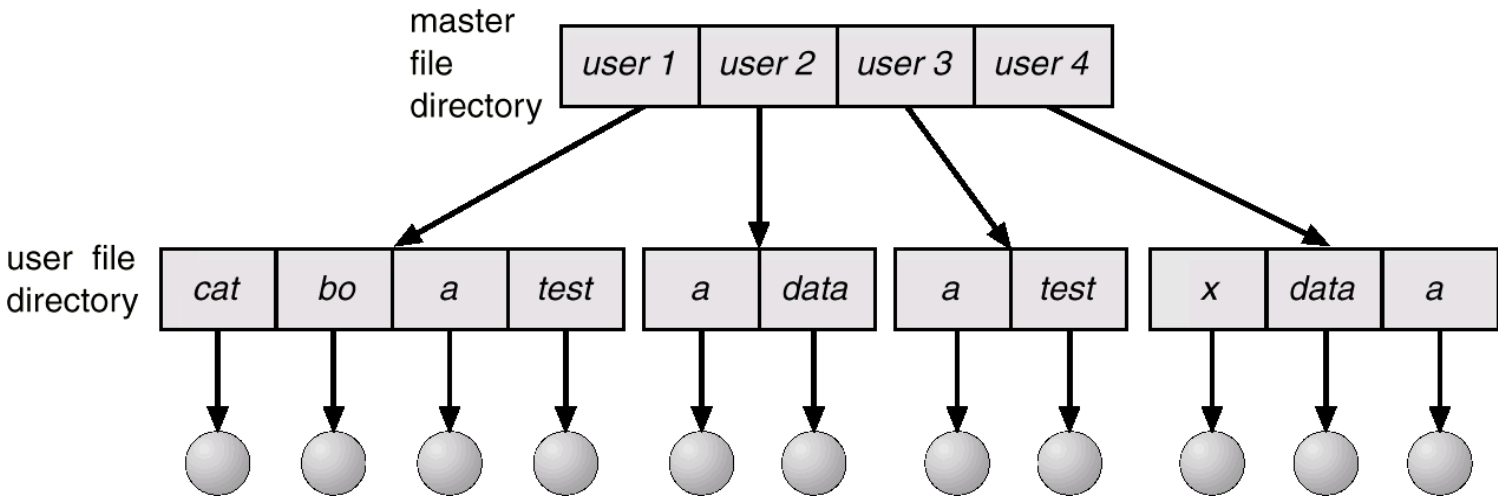
- A single directory for all users



- Naming problem
  - 충돌
  - UNIX: 255 characters
- Grouping problem

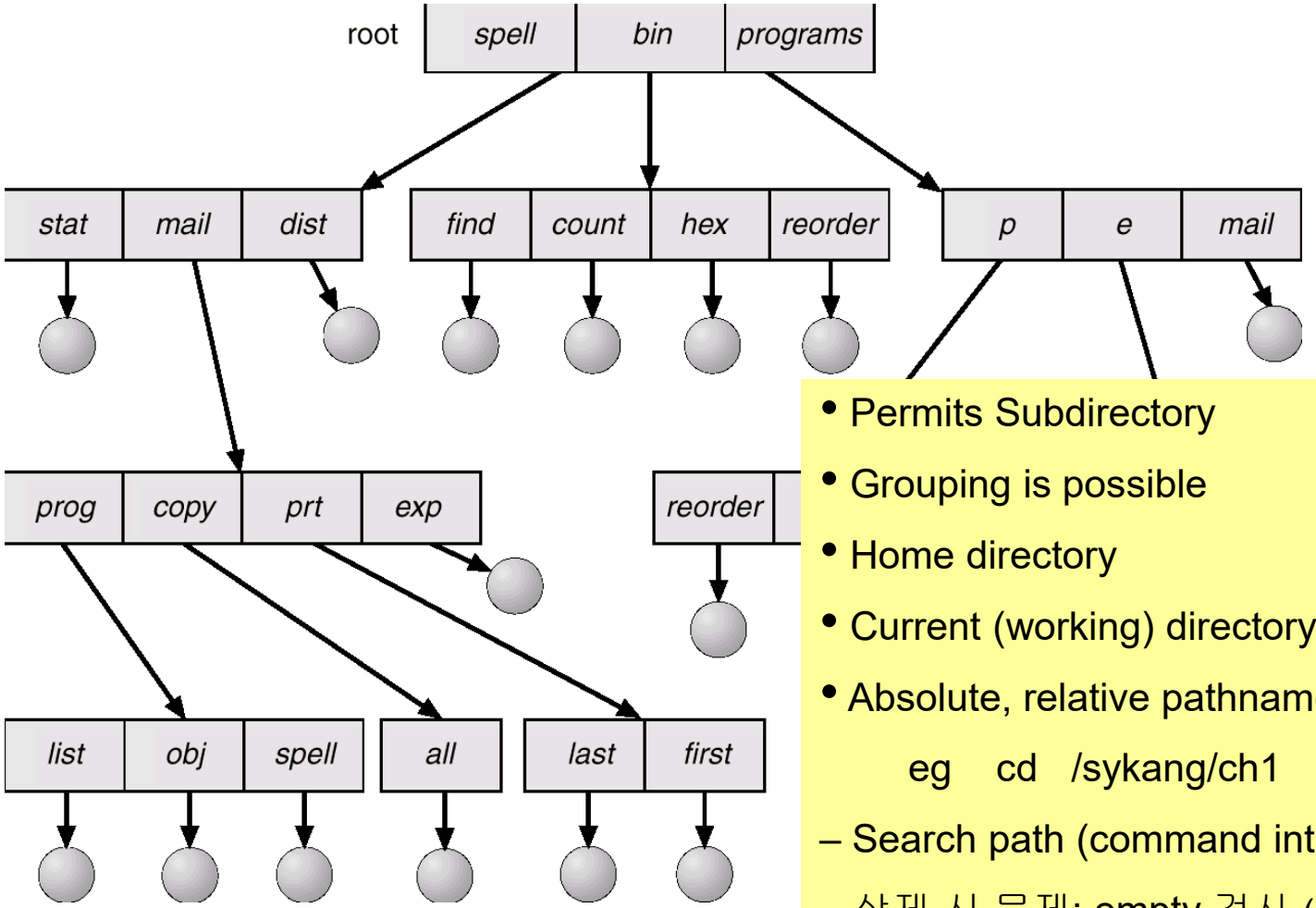
Two-Level Directory

- Separate directory for each user



- "Pathname"
- 사용자간 이름 출동 없음
- No grouping capability

Tree-Structured Directories



- Permits Subdirectory
- Grouping is possible
- Home directory
- Current (working) directory
- Absolute, relative pathname

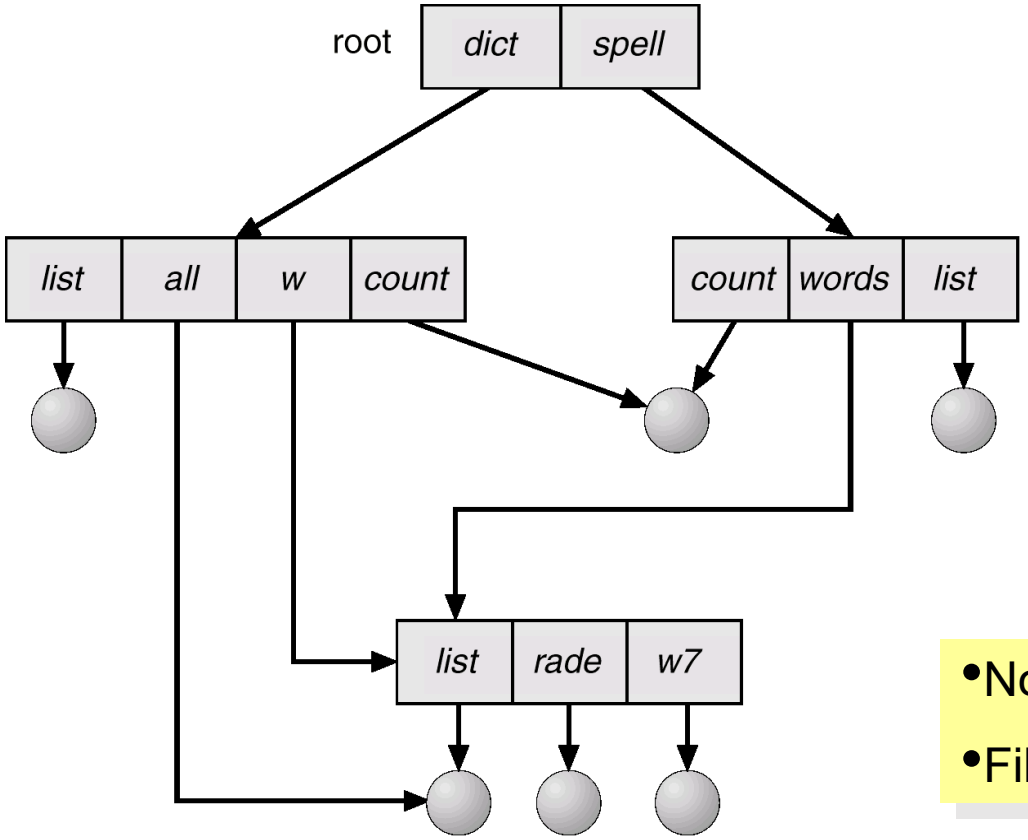
  eg cd /sykang/ch1

– Search path (command interpreter)

– 삭제 시 문제: empty 검사 (편의/위험)

Acyclic-Graph Directories

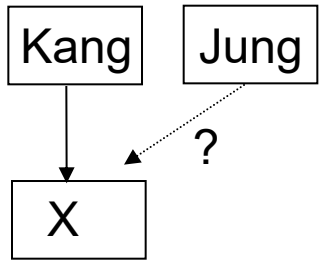
- The same {subdirectory, file} may be in two different directories



- No cycles allowed (acyclic)
- File/Directory sharing

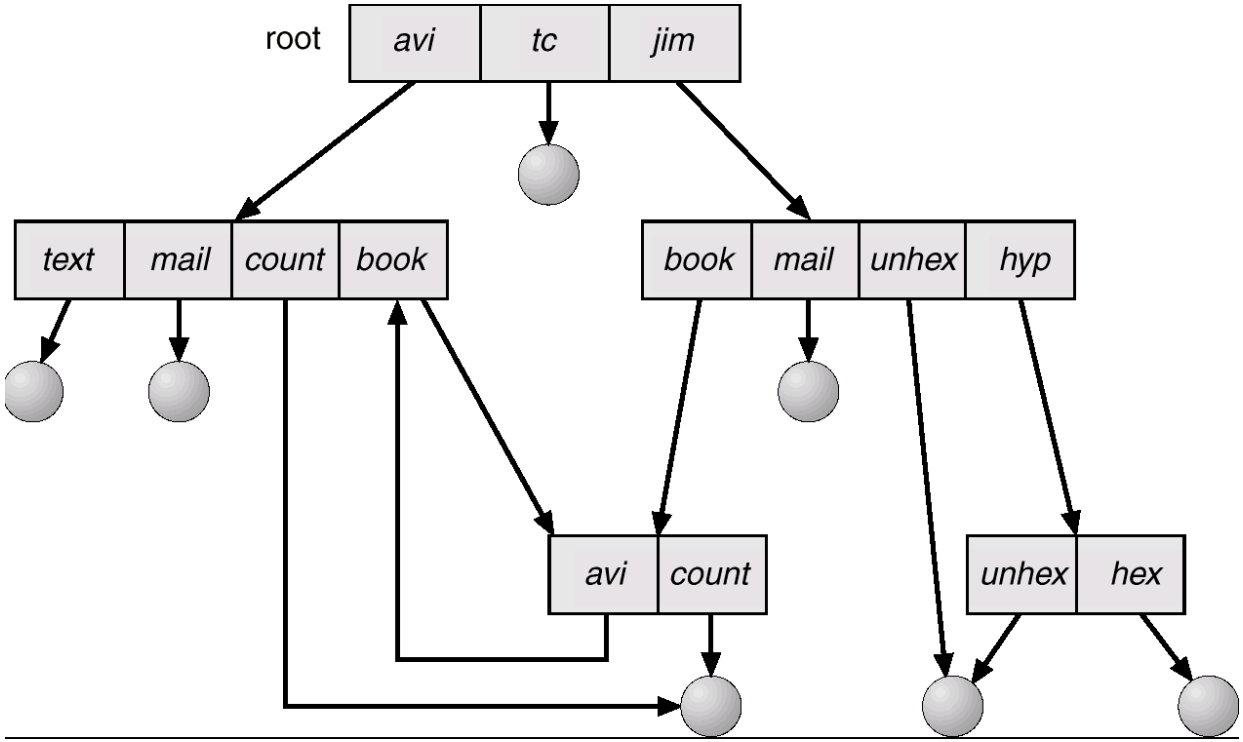
Acyclic-Graph Directories

- The same (subdirectory, file) may be in two different directories
  - → 1. traverse problem (visit same node twice) 2. Delete problem
- Scenario:
  1. Kang has subdirectory X (i.e. Kang has an entry for X)
  2. Jung need to have X below his directory
  3. UNIX – creates a "link" (a new directory entry) in Jung's directory



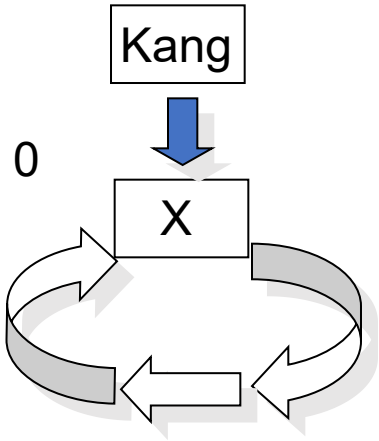
- Link type 1: "Symbolic link"
  - Link is pathname to X (indirect pointer)
  - Problem: When X is deleted from Kang, Jung's link becomes a dangling reference
    - When Jung accesses X, Just say "…. illegal file …."
- Link type 2: "Hard link"
  - Copy the directory entry in the Kang directory to Jung
    - Consistency problem when X is updated
    - How to delete X?: Maintain a reference count
      - If the reference count becomes 0 then delete X

General Graph Directory



General Graph Directory (Cont.)

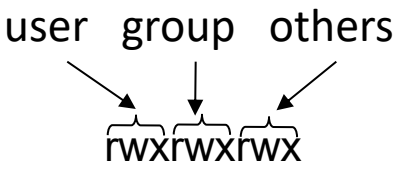
- Permits cycles
- Complex algorithms for traverse and delete
- Traverse
  - Should avoid infinite loop in the cycle
- Delete
  - Even when Kang deletes X, the reference count of X > 0 (due to self-referencing or cycle)
  - X is not deallocated
  - Needs garbage collection:
    - Mark and free those nodes that are not accessible
    - Time consuming!



- How do we guarantee no cycles?
  - Allow only links to file (not subdirectories)
  - Every time a new link is added, use a cycle detection algorithm to determine whether it is OK

Protection

- File owner/creator should be able to control:
  - what operation can be performed on the file
  - by whom
- Types of access
  - Read
  - Write
  - Execute
  - Append
  - Delete
  - List (name, attribute)
- Access Control
  - Access control matrix: too big!
  - Unix – Divide the entire users into three categories

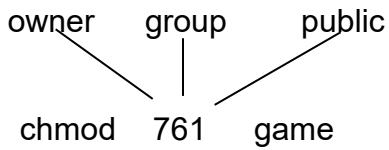


Groups

- Mode of access: read, write, execute
- Three classes of users

			RWX
a) owner access	7	⇒	1 1 1
			RWX
b) groups access	6	⇒	1 1 0
			RWX
c) public access	1	⇒	0 0 1

- Ask manager to create a group (unique name), say G, and add some users to the group
- For a particular file (say *game*) or subdirectory, define an appropriate access



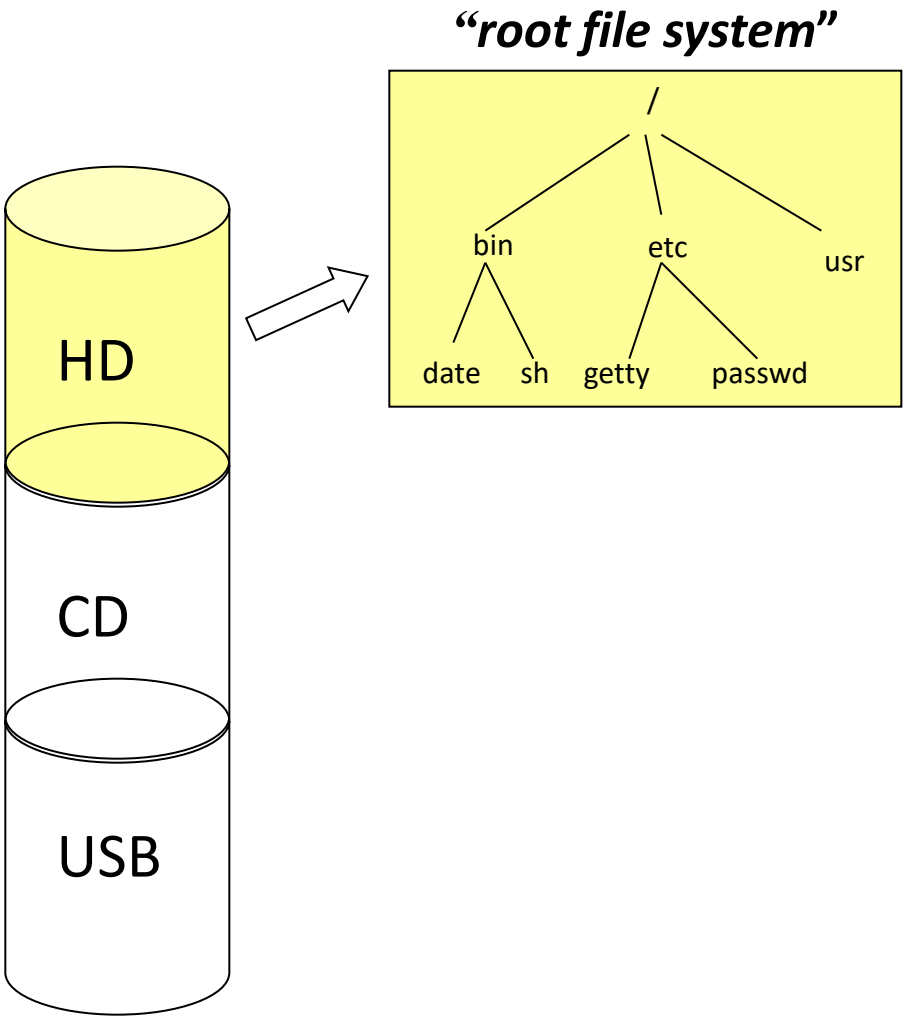
- Attach a group to a file: chgrp G game

File-System Structure

- File system generally resides on secondary storage (disks)
- File system organized into layers
- open() system call
  - Load the metadata of the specified file from disk to memory
    - Needs directory structure search
  - Open-file table
    - Stores metadata of in-memory (opened) files
  - File descriptor (file handle, file control block)
    - Index into open-file table
  - Why?
    - Directory search is expensive
    - So many files in disk

eg: /a/b/c/d/e.hwp
disk I/O's

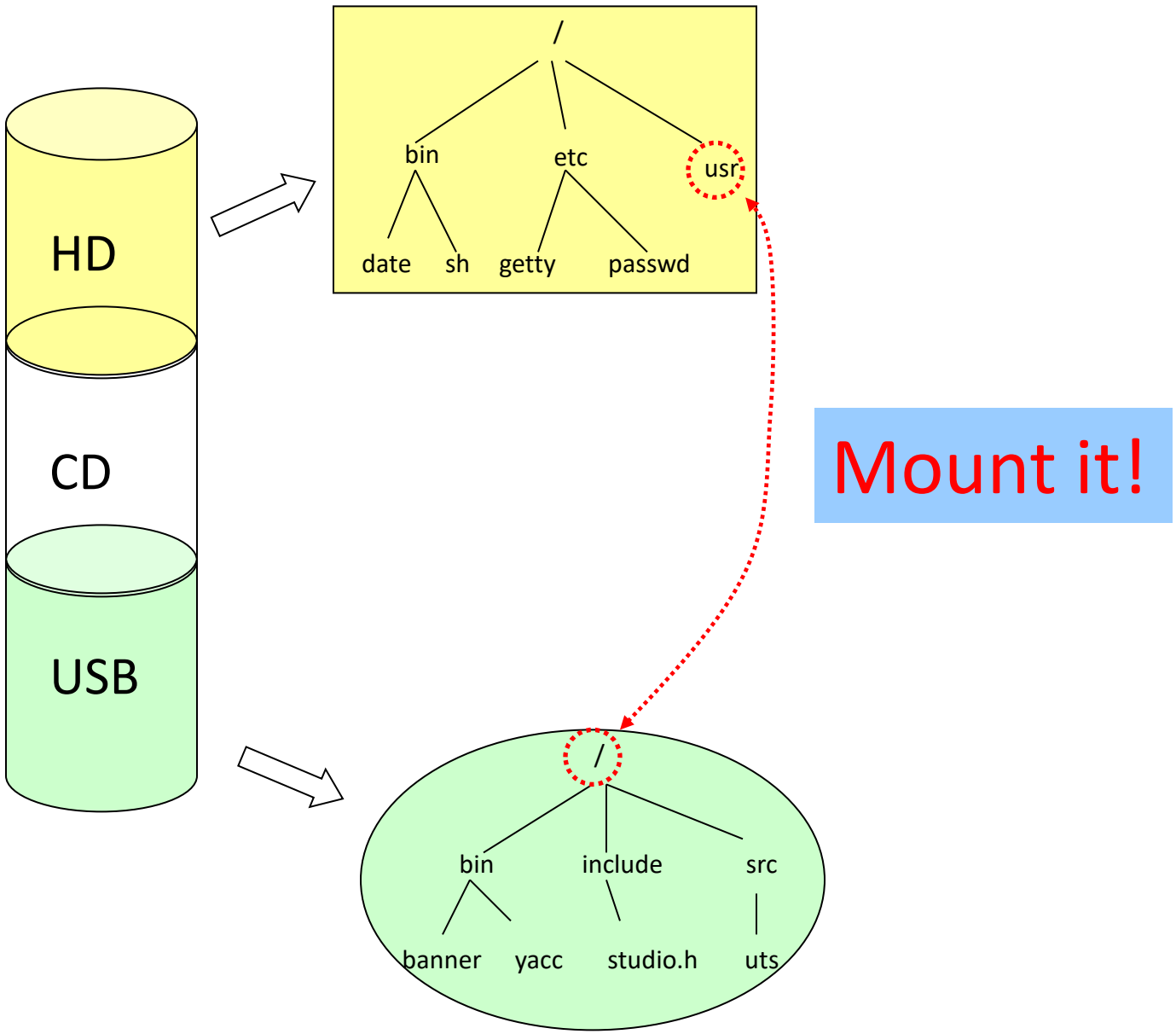
Mounting File System



Now
all files under
root file system
can be accessed

But
how do we access files
in other FS?

Mounting File System

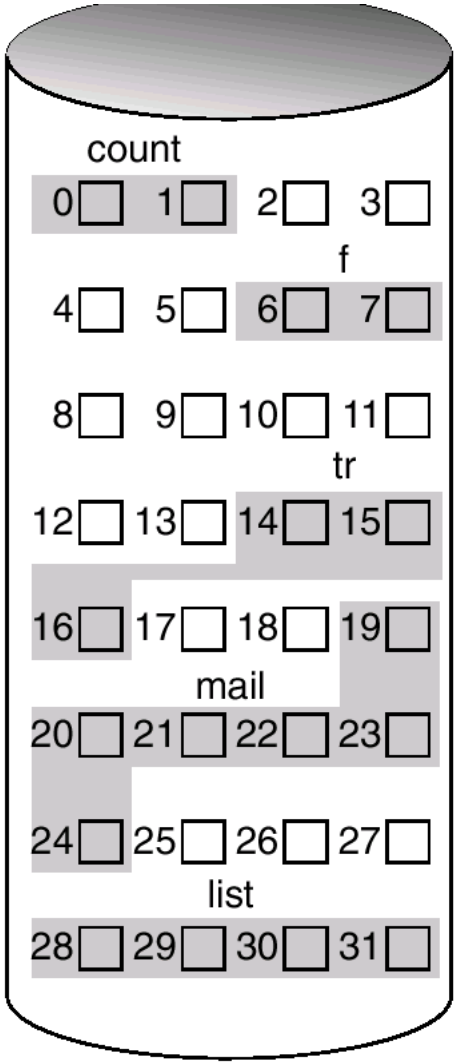


Allocation of File Data in Disk

Contiguous Allocation

- Each file occupies a set of contiguous blocks on the disk
- Simple
  - Only starting location (block #) and length (# of blocks) are required
- Wasteful of space
  - Dynamic storage-allocation → external fragmentation → numerous small holes
- Difficulty in File growth
  - File 생성시 얼마나 큰 hole을 배당할 것인가?
  - Grow 가능 vs 낭비 (internal fragmentation)
- Fast I/O
  - 한번의 seek/rotation으로 많은 data transfer
  - Realtime file 용으로, 또는
  - 이미 run 중이던 process의 swapping 용 (entire R/W 보장)

Contiguous Allocation of Disk Space

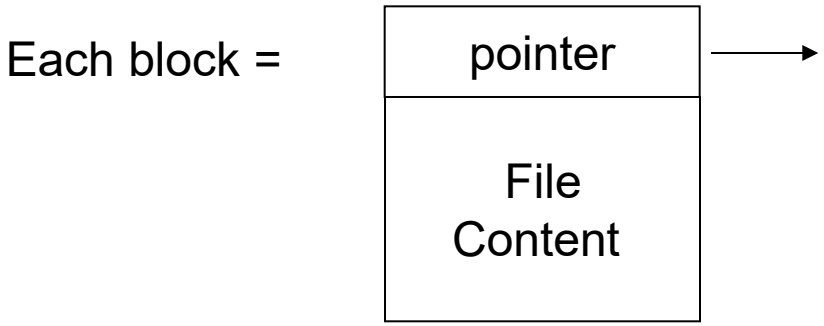


directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

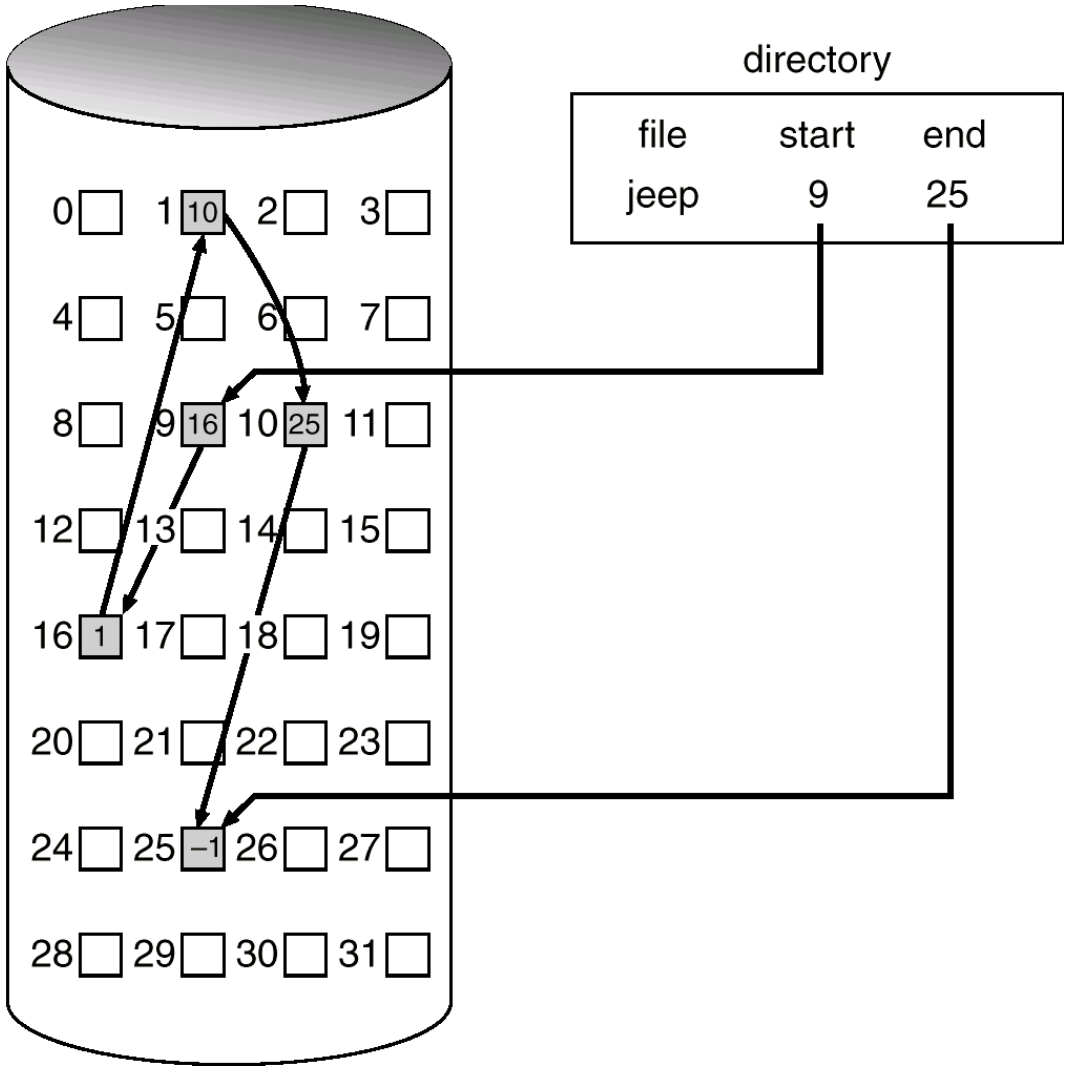
Linked Allocation

- Each file is a linked list of disk blocks
- Blocks may be scattered anywhere on the disk



Linked Allocation (Cont.)

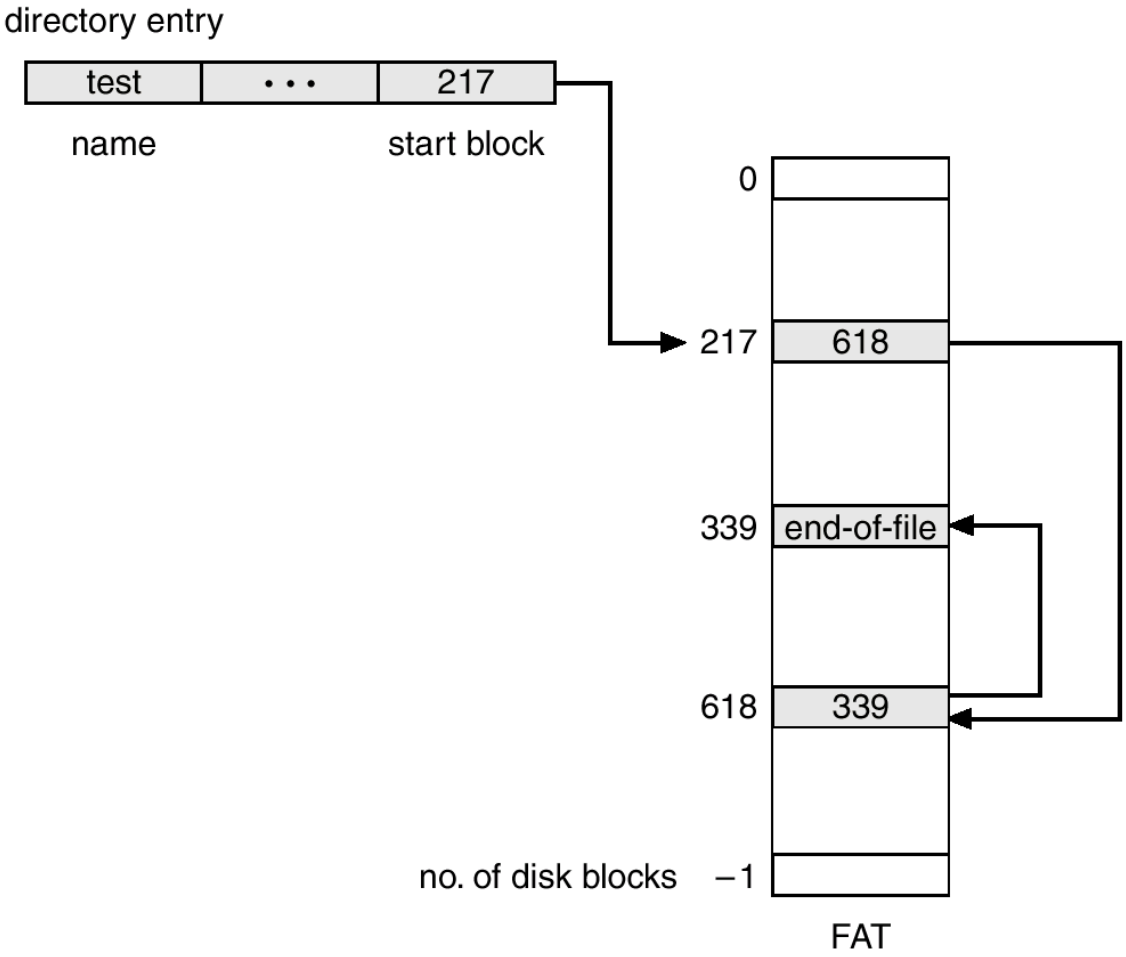
- Allocate as needed, link together
  - e.g., file starts at block 9



Linked Allocation (Cont.)

- Simple – need only starting address
- Free-space management system – no waste of space
- Drawbacks
  - No random access
  - Disk I/O efficiency (매 sector I/O 마다 seek/rotation)
  - Reliability 문제
    - Bad sector로 인한 pointer 유실 ➔ 많은 부분을 잃음
  - Space for pointers (0.78% for pointers – 512 B/sector, 4 B/pointer)
- Variation: File-allocation table (FAT)
  - Disk-space allocation used by MS Windows and OS/2

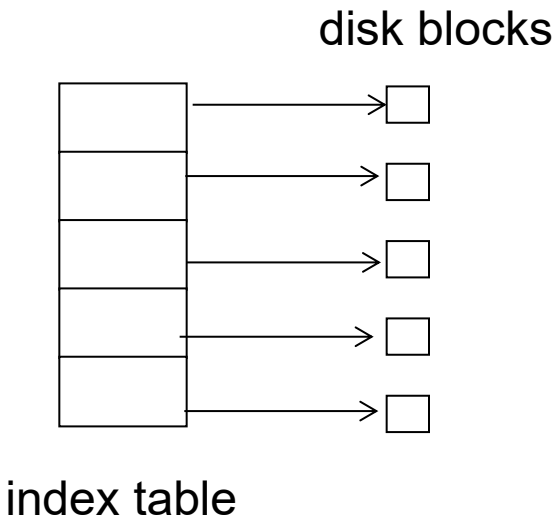
File-Allocation Table



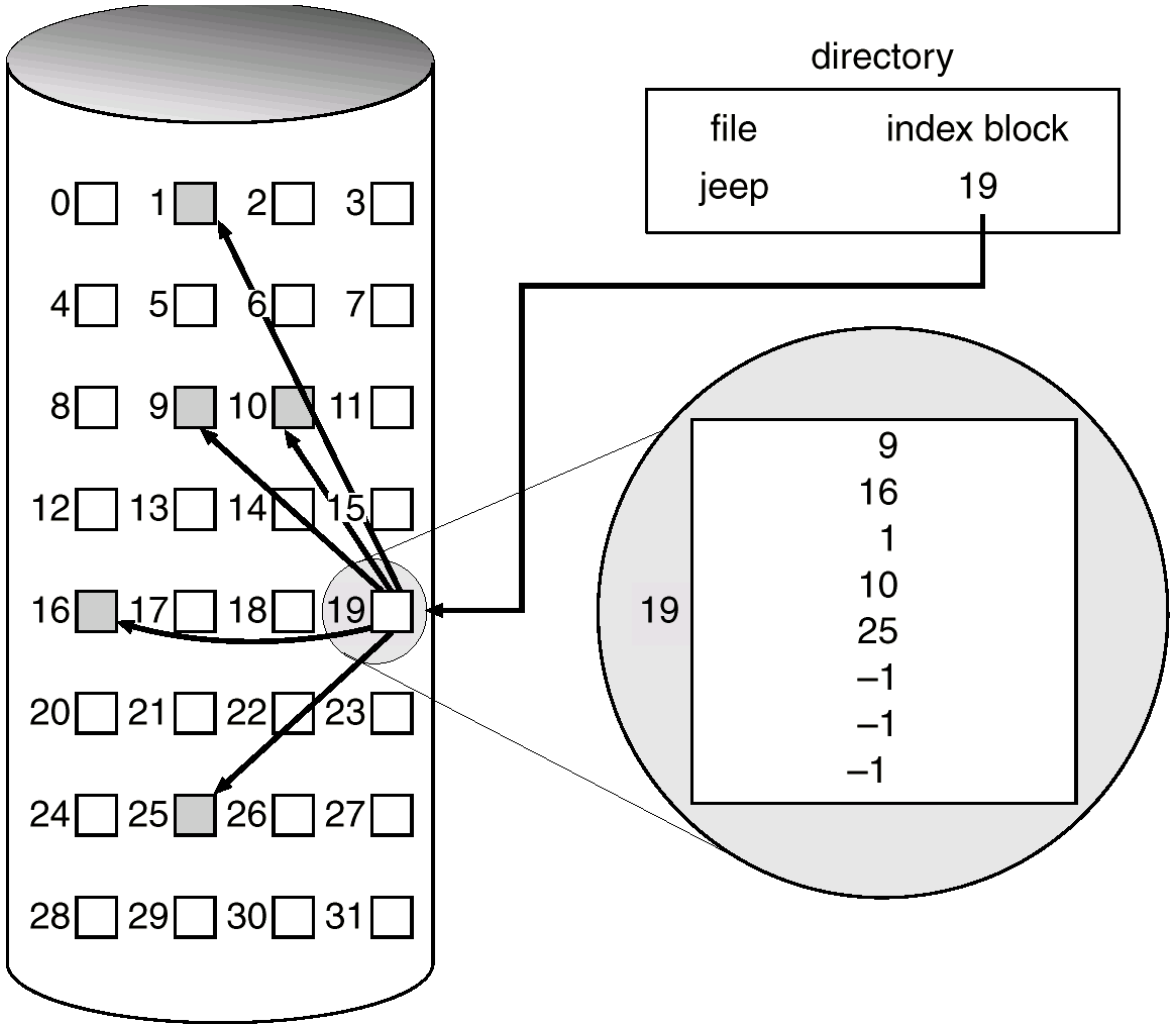
- Faster random access

Indexed Allocation

- Each file has its own index block
- Index block is an array of pointers to disk blocks
- Speed up direct access
- Logical view



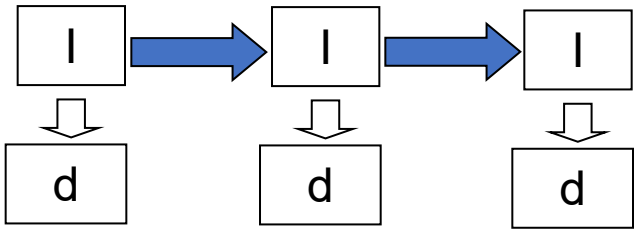
Example of Indexed Allocation



Indexed Allocation – Mapping (Cont.)

- What if file is too large?
  - One disk block is too small for index table

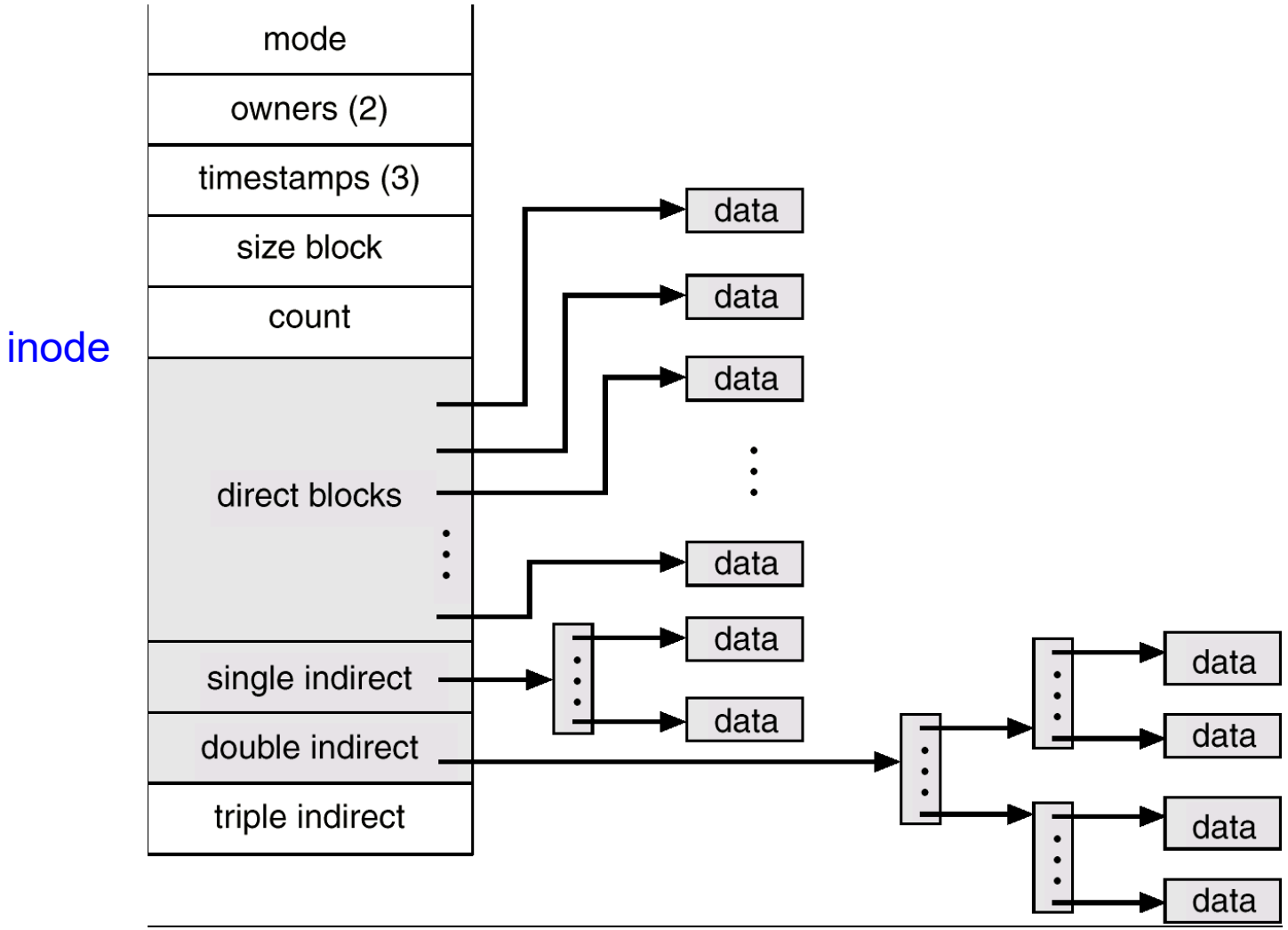
1. Linked scheme – Link blocks of index table (no limit on size)



2. Two-level index

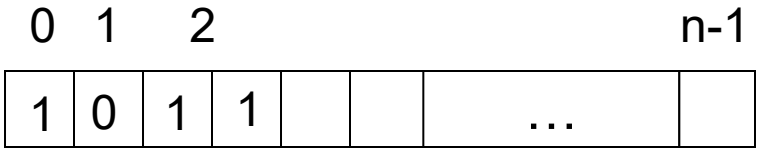
3. Combined (e.g., UNIX)

Combined Scheme: UNIX (4K bytes per block)



Free-Space Management

- Bit map or bit vector



$$\text{bit}[i] = \begin{cases} 1 \Rightarrow \text{block}[i] \text{ free} \\ 0 \Rightarrow \text{block}[i] \text{ occupied} \end{cases}$$

  - Block number calculation for the first free block

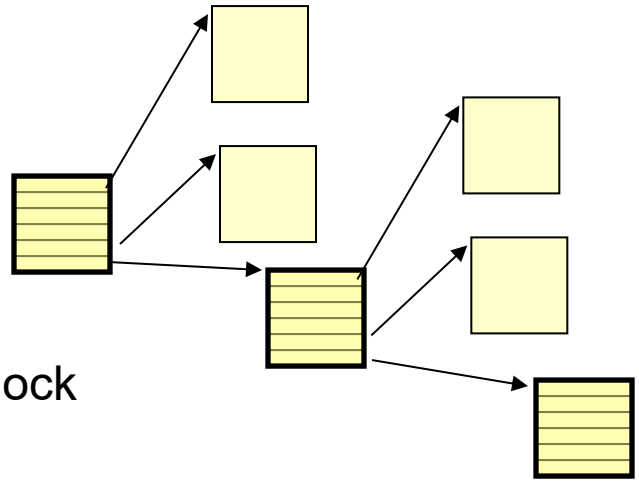
    (number of bits per word) *
    (number of 0-value words) +
    offset of first 1 bit

    Bit map requires extra space

    Easy to get contiguous files

Free-Space Management (Cont.)

- Linked list
  - Link all the free blocks (free list)
  - Traversing this list is slow (disk I/O), but is not frequent
  - Cannot get contiguous space easily
  - No waste of space
- Grouping
  - Modification of linked list approach
  - First free block has *n* pointers
    - *n*-1 pointers point to free blocks
    - Last pointer points to the same kind of block



- Counting
  - Keep track of (first free block, # of contiguous free blocks)
  - Generally, several contiguous blocks are allocated & freed together

Performance

- Disk cache
  - Separate section of main memory for frequently used data blocks
- Free-behind and Read-ahead
  - Techniques to optimize sequential access
  - Next block request 할 때 현 block을 free (Free-behind)
  - A requested page and subsequent pages are read and cached (Read-ahead)
- Virtual disk (RAM disk)
  - Improve PC performance by dedicating section of memory
- Directory entry cache (dentry cache)
  - Keep frequently used directory entries in memory

Recovery

- Consistency checker
  - Some part of file system is kept in memory
  - What if system crashes? Not all the info can be saved to disk
  - If directory or file control block (metadata) are lost?
  - At boot time -- compare data in directory structure with data blocks on disk, and tries to fix inconsistencies
- Use system programs to *back up* data from disk to another storage device (floppy disk, magnetic tape)
- Recover lost file or disk by *restoring* data from backup

Final Remarks

- So …
  - “File System” means data structure on disk
  - “logical formatting” – initializing this data structure
  - Accessing a disk file is OS dependent operation